

## Supplementary Material for High-Throughput and Low-Latency DrDoS Detection Using Quantized ONNX Models

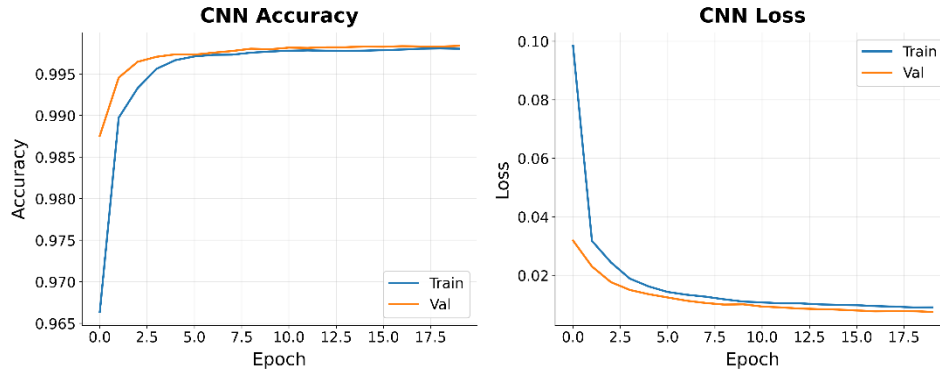


Figure S1: Validation-Training Loss and Accuracy Graphs for the CNN Model

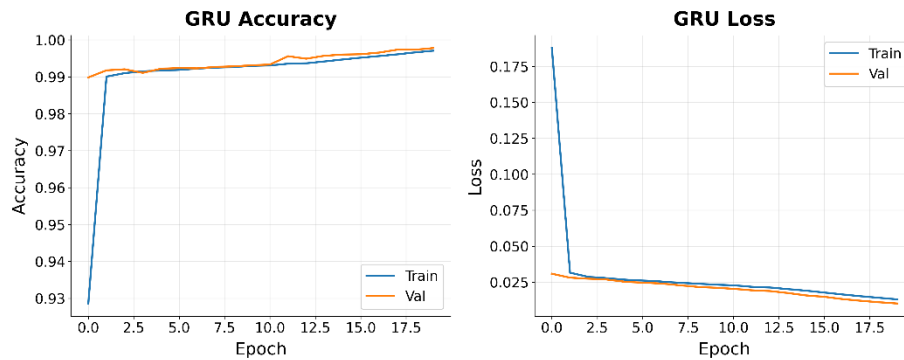


Figure S2: Validation-Training Loss and Accuracy Graphs for the GRU Model

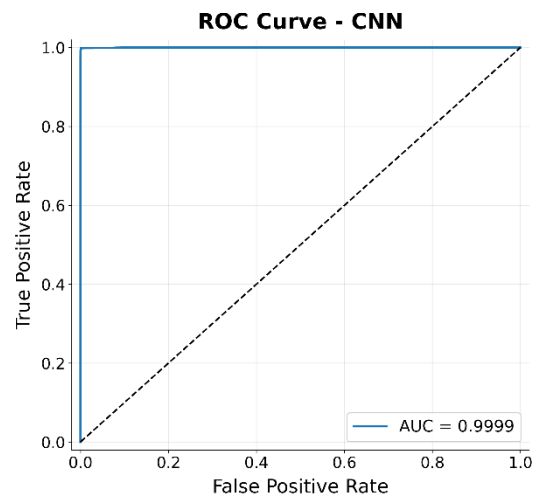
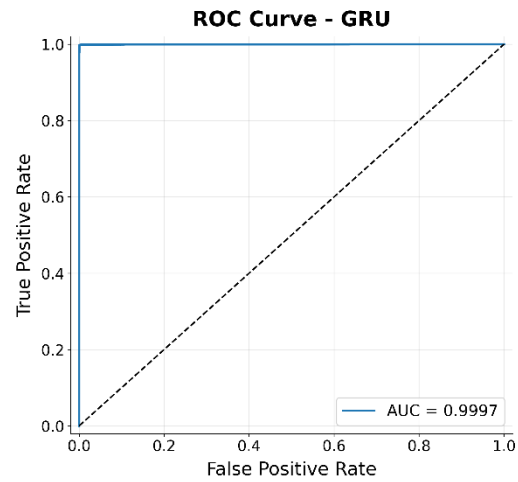
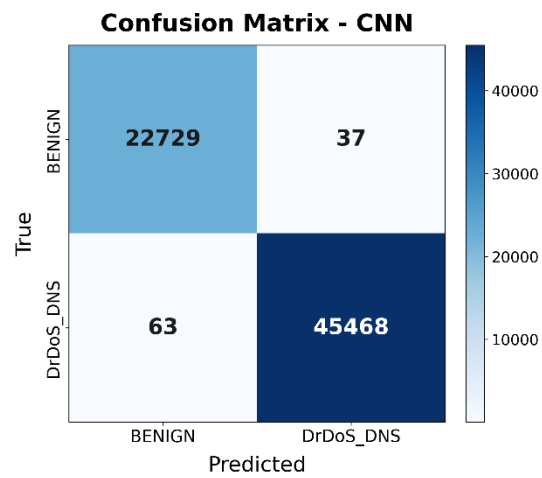


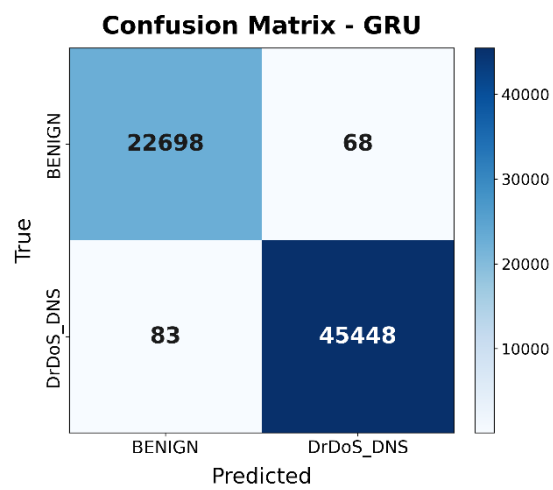
Figure S3: Test ROC Curve for the CNN Model



**Figure S4: Test ROC Curve for the GRU Model**



**Figure S5: Confusion Matrix for the CNN Model**



**Figure S6: Confusion Matrix for the GRU Model**

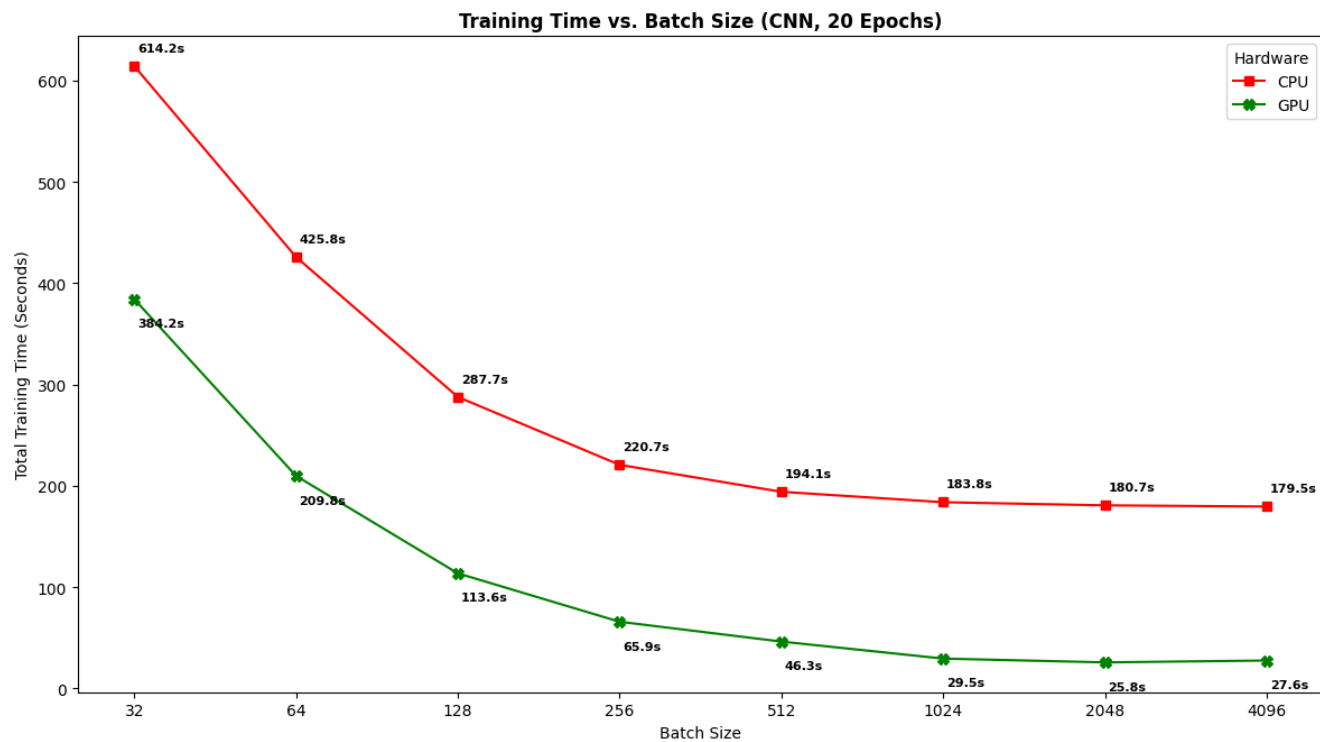


Figure S7: Training Time vs Batch Size (CNN)

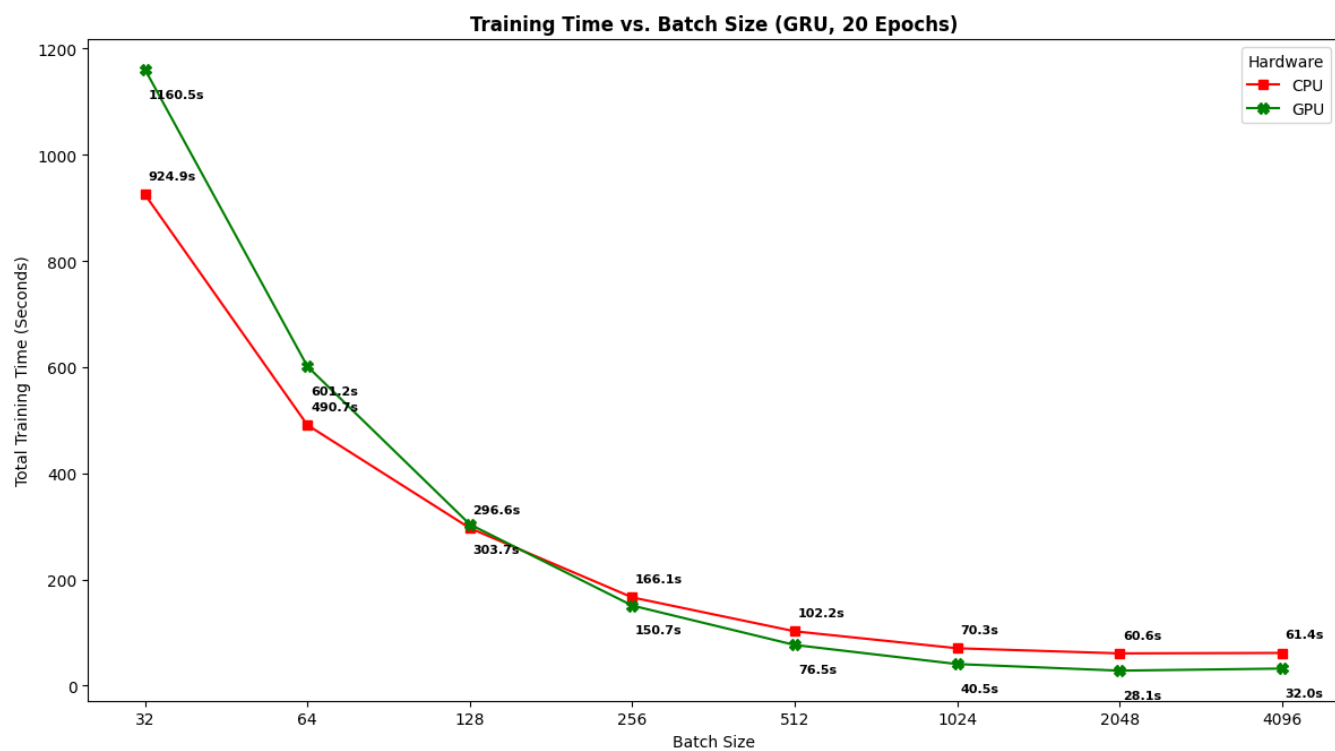


Figure S8: Training Time vs Batch Size (GRU)

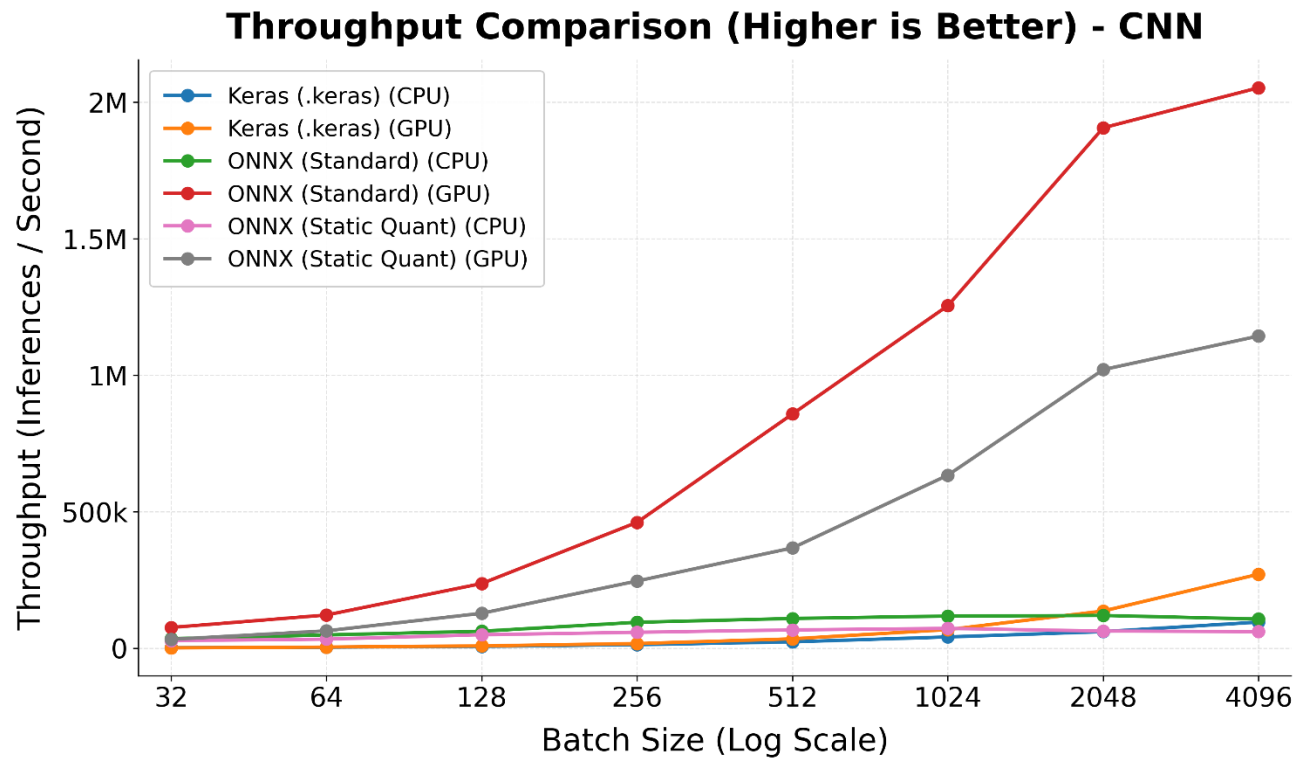


Figure S9: Throughput vs Batch Size (CNN)

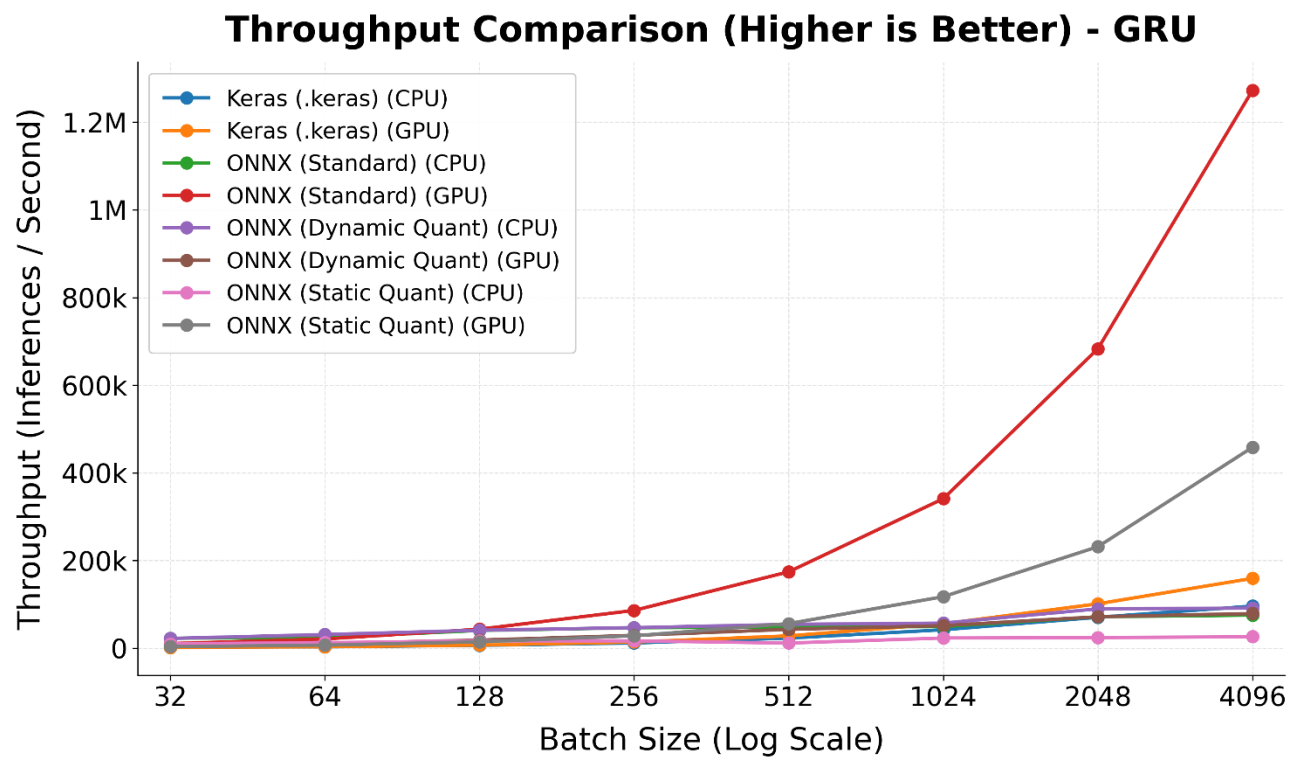


Figure S10: Throughput vs Batch Size (GRU)

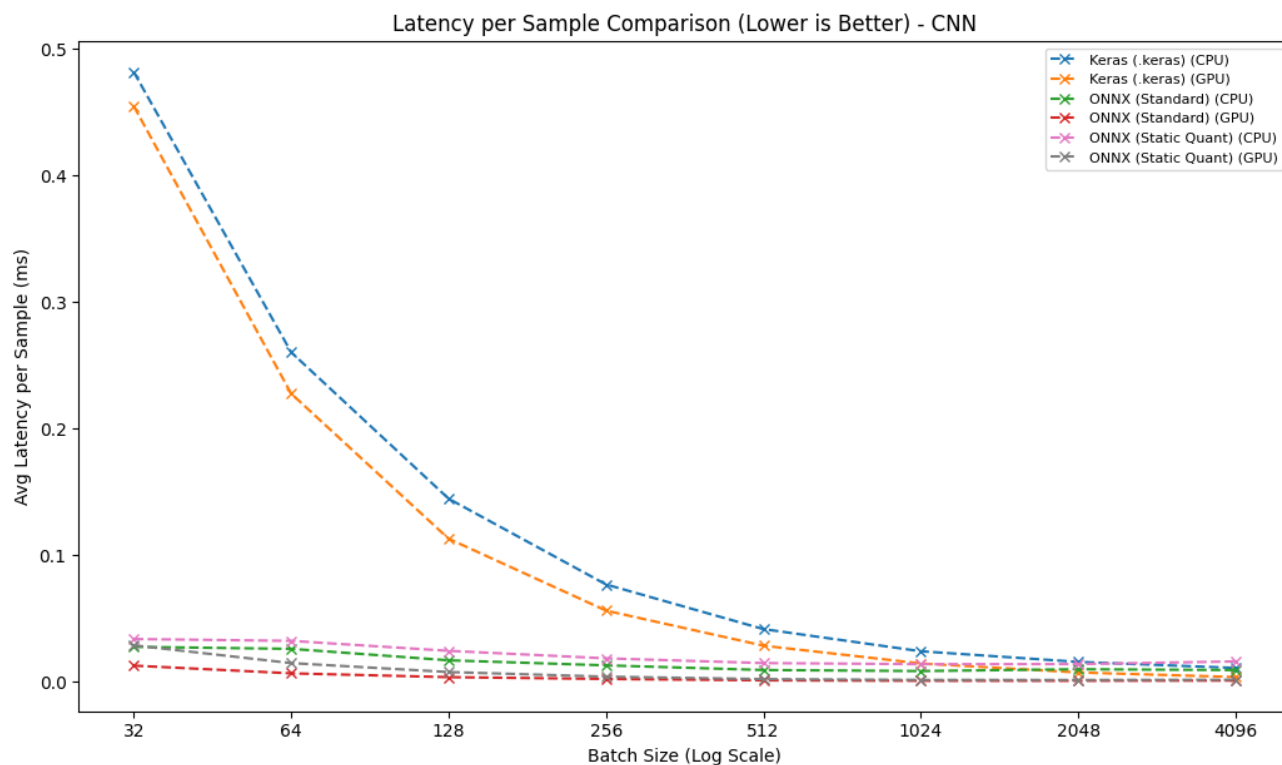


Figure S11: Latency vs Batch Size (CNN)

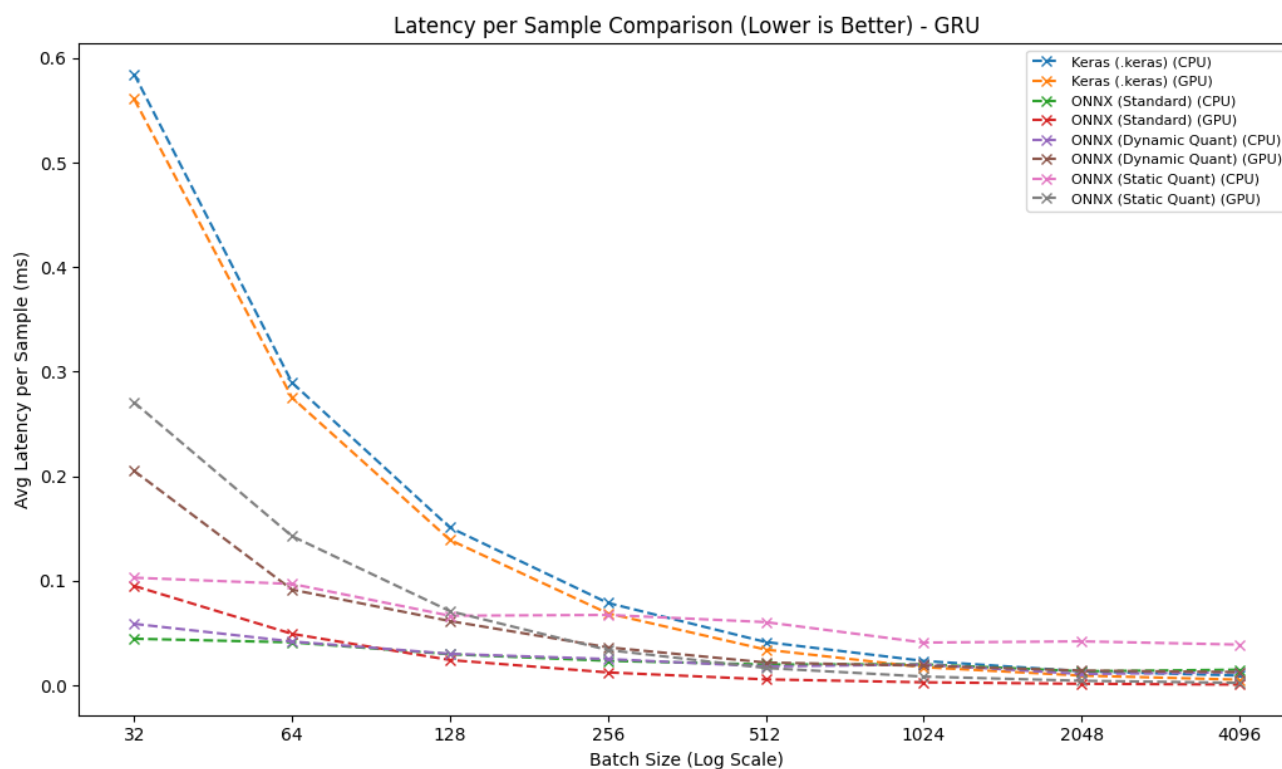


Figure S12: Latency vs Batch Size (GRU)

**Table S1: CNN Model Comparison Summary.**

| Model                 | Environment | Inference Time (μs) | File size (MB) |
|-----------------------|-------------|---------------------|----------------|
| Keras                 | CPU         | 10.7489             | 0.7086         |
| Keras                 | GPU         | 3.5619              | 0.7086         |
| ONNX                  | CPU         | 8.2803              | 0.226          |
| ONNX                  | GPU         | 0.5917              | 0.226          |
| Static Quantized ONNX | CPU         | 13.7391             | 0.1056         |
| Static Quantized ONNX | GPU         | 1.2097              | 0.1056         |

**Table S2: GRU Model Comparison Summary.**

| Model                  | Environment | Inference Time (μs) | File size (MB) |
|------------------------|-------------|---------------------|----------------|
| Keras                  | CPU         | 9.3467              | 0.556          |
| Keras                  | GPU         | 5.3923              | 0.556          |
| ONNX                   | CPU         | 13.4195             | 0.2625         |
| ONNX                   | GPU         | 0.7397              | 0.2625         |
| Dynamic Quantized ONNX | CPU         | 10.8018             | 0.2534         |
| Dynamic Quantized ONNX | GPU         | 12.6615             | 0.2534         |
| Static Quantized ONNX  | CPU         | 38.9413             | 0.7065         |
| Static Quantized ONNX  | GPU         | 2.3361              | 0.7065         |

## LIBRARY VERSIONS (reproducibility)

- python : 3.12.13
- platform : Linux-6.6.122+-x86\_64-with-glibc2.35
- numpy : 2.0.2
- pandas : 2.2.2
- scikit\_learn : 1.6.1
- scipy : 1.16.3
- matplotlib : 3.10.0
- seaborn : 0.13.2
- tensorflow : 2.20.0
- keras : 3.13.2
- onnx : 1.22.0
- tf2onnx : 1.17.0
- onnxsim : 0.6.5
- protobuf : 5.29.6
- onnxruntime\_gpu (GPU inference) : 1.20.1
- onnxruntime\_cpu (CPU inference / CNN dynamic-quant): 1.27.0
- cuda : 12.5.1
- cudnn : 9
- gpu : A100 [PhysicalDevice(name='/physical\_device:GPU:0', device\_type='GPU')]
- ONNX Runtime: GPU=onnxruntime-gpu 1.20.1 | CPU=onnxruntime 1.27.0 (iki ayrı faz)

- Random seeds: global=42 | split=42 | downsample=27 | variance=1000-1029

All experiments were conducted on Google Colab using an NVIDIA A100 GPU (Linux 6.6.122, x86\_64, glibc 2.35; CUDA 12.5.1, cuDNN 9). The software stack was: Python 3.12.13, TensorFlow 2.20.0 (Keras 3.13.2), NumPy 2.0.2, pandas 2.2.2, scikit-learn 1.6.1, SciPy 1.16.3, matplotlib 3.10.0, seaborn 0.13.2, ONNX 1.22.0, tf2onnx 1.17.0, onnxsim 0.6.5, and protobuf 5.29.6. ONNX inference was evaluated with two ONNX Runtime builds in separate sessions: ONNX Runtime-GPU 1.20.1 for GPU benchmarks—executed in a TensorFlow-free session to avoid GPU-memory contention—and ONNX Runtime 1.27.0 (CPU build) for CPU benchmarks, including the CNN dynamic-quantized model whose ConvInteger operator is implemented only in the CPU build. All random seeds were fixed to ensure determinism: a global seed of 42 (NumPy, Python random, and TensorFlow), `random_state = 42` for the stratified train/test split, `random_state = 27` for downsampling.